

Aim:

A logistics company in Bengaluru deploys autonomous drones to deliver medical supplies across multiple Indian cities.

- Each drone must start from the headquarters (HQ).
- It must visit every city exactly once.
- Finally, it must return to HQ while minimizing total travel cost (fuel, distance, or time).

This is essentially the Travelling Salesman Problem (TSP), and the company's system uses Dynamic Programming (DP) to compute the minimum travel cost.

Input Format:

- The first line contains an integer n , representing the number of cities.
- The next n lines each contain n space-separated integers representing the travel cost matrix, where each value indicates the cost of traveling from one city to another.

Output Format:

- Print a single integer representing the minimum possible tour cost to visit all cities exactly once, starting and ending at city 0.

Note: Partial code is provided. Complete the **tsp()** function to find the minimum tour cost.

Source Code:

droneDeliveryCost.c

```
#include <stdio.h>
#include <limits.h>
#define MAX 15
int n;
int cost[MAX][MAX];
int dp[1 << MAX][MAX];
// Declare variables required for the TSP solution.

int tsp(int mask, int pos) {

    // Implement the DP logic to find the minimum TSP tour cost.
    if(mask== (1 << n)-1){
        return cost[pos][0];
    }
    if(dp[mask][pos]!= -1){
        return dp[mask][pos];
    }
    int ans=INT_MAX;

    for(int city=0;city<n;city++){
        if((mask& (1<<city))==0){
            int newCost=cost[pos][city]+ tsp(mask|(1<<city),city);
            if(newCost<ans){
                ans=newCost;
            }
        }
    }
}
```

```

    }
    return dp[mask][pos]=ans;
}

int main() {
    scanf("%d", &n);
    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);
    for(int i = 0; i < (1 << MAX); i++)
        for(int j = 0; j < MAX; j++)
            dp[i][j] = -1;
    printf("%d\n", tsp(1, 0));
    return 0;
}

```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
80

Test Case - 2
User Output
3
0 5 10
5 0 3
10 3 0
18